

Docker

From Zero to Production

A complete guide for interns and beginners — understanding containerization, Docker commands, Compose, and deploying Django applications.

[What is Docker](#)[Core Concepts](#)[Installation](#)[Commands](#)[Dockerfile](#)[Docker Compose](#)[Django + Docker](#)[Production](#)[Checklist](#)

1 What is Docker & Why Use It?

Docker is a tool that packages your application and everything it needs (Python version, libraries, environment variables, OS dependencies) into a single unit called a **container**.

🤔 THE CLASSIC PROBLEM

"It works on my machine!" — Every developer ever. Your app runs fine locally but breaks on the server because Python version is different, or a library is missing, or the OS is different.

✅ DOCKER'S SOLUTION

Docker packages EVERYTHING together. Now the same container runs identically on your laptop, your teammate's machine, and the production server.

✗ Without Docker

- Manual setup on every machine
- Python version conflicts
- Library version mismatches
- "Works on my machine" issues
- Hard to scale
- Complex deployment process

✅ With Docker

- One command: `docker-compose up`
- Same environment everywhere
- All dependencies locked in
- Easy team collaboration
- Scale with one command
- Simple, repeatable deployments

🏠 Real-World Analogy

Think of Docker as a **shipping container**. Before containers, loading cargo on ships was messy — different shapes, sizes, handling requirements. Shipping containers standardized everything. Similarly, Docker containers standardize how applications are packaged and shipped.

2 Core Concepts

Image

A **read-only template** with instructions to create a container. Like a blueprint or a recipe. Images are built from a `Dockerfile`. Once built, they don't change.

IMAGE ANALOGY

Image = Class in Python. Container = Object (instance) of that class. You can create multiple containers from one image, just like multiple objects from one class.

Container

A **running instance** of an image. Containers are isolated from each other and from the host system. You can start, stop, delete containers freely — the image stays safe.

Dockerfile

A text file with step-by-step instructions to build a Docker image. It's like a recipe — it tells Docker exactly how to set up the environment for your application.

Docker Hub

An online registry (like GitHub for code) where Docker images are stored. Official images for Python, PostgreSQL, Redis, Nginx are all available here at **hub.docker.com**.

Docker Compose

A tool to define and run **multi-container** applications. Instead of starting Django, PostgreSQL, and Redis separately with long commands, you define everything in one `docker-compose.yml` file and run it all with `docker-compose up`.

 User / Browser
↓
Nginx Container
↓
Django Container
↔
PostgreSQL Container
↔
Redis Container
Docker Compose manages all containers together as one application

3 Installation

Windows

Download and install **Docker Desktop** from docker.com/products/docker-desktop. It includes Docker Engine, Docker CLI, and Docker Compose — everything in one installer.

WINDOWS REQUIREMENT

Docker Desktop requires WSL 2 (Windows Subsystem for Linux). The installer will guide you through enabling it. Make sure Windows 10/11 is updated.

macOS

Download Docker Desktop for Mac from [docker.com](https://www.docker.com). Available for both Intel and Apple Silicon (M1/M2/M3) chips.

Linux (Ubuntu)

BASH

```
# Update packages
sudo apt-get update

# Install Docker
sudo apt-get install docker-ce docker-ce-cli containerd.io

# Install Docker Compose
sudo apt-get install docker-compose-plugin

# Add your user to docker group (no sudo needed)
sudo usermod -aG docker $USER

# Verify installation
docker --version
docker compose version
```

Verify Installation

BASH

```
# Run hello-world to test
docker run hello-world

# You should see: "Hello from Docker!"
# This means Docker is working correctly.
```

4 Essential Docker Commands

Image Commands

Command	Description
docker images	List all downloaded images
docker pull python:3.11	Download an image from Docker Hub
docker build -t myapp .	Build an image from Dockerfile in current directory
docker rmi myapp	Remove/delete an image
docker image prune	Remove all unused images

Container Commands

Command	Description
docker run myapp	Create and start a container from image
docker run -d myapp	Run in background (detached mode)
docker run -p 8000:8000 myapp	Map port: host:container
docker ps	List running containers
docker ps -a	List ALL containers (including stopped)
docker stop container_id	Stop a running container
docker start container_id	Start a stopped container
docker rm container_id	Delete a container
docker logs container_id	View container logs
docker logs -f container_id	Follow logs in real-time
docker exec -it container_id bash	Open terminal inside container

Cleanup Commands

BASH

```
# Remove all stopped containers
docker container prune

# Remove all unused images
docker image prune

# Nuclear option - remove everything unused
docker system prune -a

# Check disk usage
docker system df
```

5 Writing a Dockerfile

A Dockerfile is a set of instructions to build your image. Each instruction creates a new "layer" in the image.

Dockerfile Instructions Explained

Instruction	Description
FROM	Base image to start from (e.g., python:3.11-slim)
WORKDIR	Set working directory inside container
COPY	Copy files from host to container
RUN	Execute a command during image build
ENV	Set environment variables
EXPOSE	Document which port the app uses (informational)
CMD	Command to run when container starts

Django Dockerfile — Step by Step

DOCKERFILE

```
# Step 1: Choose base image
# python:3.11-slim = Python 3.11 on minimal Linux (smaller size)
FROM python:3.11-slim

# Step 2: Prevent Python from creating .pyc files
ENV PYTHONDONTWRITEBYTECODE=1

# Step 3: Prevent Python from buffering stdout/stderr
ENV PYTHONUNBUFFERED=1

# Step 4: Set working directory inside container
WORKDIR /app

# Step 5: Copy requirements FIRST (for Docker layer caching)
# This way, if only code changes, pip install is not re-run
COPY requirements.txt .

# Step 6: Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Step 7: Copy the rest of your project
COPY . .

# Step 8: Expose port (documentation only, not actual port mapping)
EXPOSE 8000
```

```
# Step 9: Command to run when container starts
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

🔗 WHY COPY REQUIREMENTS.TXT FIRST?

Docker caches each layer. If you copy requirements.txt first and run pip install, Docker won't re-run pip install unless requirements.txt changes. This makes builds much faster.

🔗 .dockerignore File

Just like .gitignore, this tells Docker what NOT to copy into the image:

.DOCKERIGNORE

```
.git
.gitignore
__pycache__
*.pyc
*.pyo
.env
venv/
env/
*.egg-info
.pytest_cache
media/
staticfiles/
```

6 Docker Compose

Docker Compose lets you define and manage **multiple containers** as a single application. Instead of running 4 separate docker run commands for Django, PostgreSQL, Redis, and Nginx — you write one YAML file and run one command.

📄 docker-compose.yml Syntax

DOCKER-COMPOSE.YML

```
version: '3.8'

services:

  # --- Django Application ---
  web:
    build: .                # Build from Dockerfile in current dir
    command: python manage.py runserver 0.0.0.0:8000
    ports:
      - "8000:8000"         # host:container port mapping
    volumes:
      - ./app               # Sync local code changes to container
    env_file:
      - .env                # Load environment variables
    depends_on:
      - db                  # Start db before web
      - redis
    # --- PostgreSQL Database ---
  db:
    image: postgres:15      # Official image from Docker Hub
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
```

```

    volumes:
      - postgres_data:/var/lib/postgresql/data # Persist data

# --- Redis Cache ---
redis:
  image: redis:7-alpine # Lightweight Redis image

# Named volumes - data persists even if container is removed
volumes:
  postgres_data:

```

➤ Docker Compose Commands

Command	Description
<code>docker-compose up</code>	Start all services (foreground)
<code>docker-compose up -d</code>	Start all services in background
<code>docker-compose up --build</code>	Rebuild images before starting
<code>docker-compose down</code>	Stop and remove all containers
<code>docker-compose down -v</code>	Also remove volumes (wipes database!)
<code>docker-compose logs web</code>	View logs for web service
<code>docker-compose ps</code>	List all compose services status
<code>docker-compose exec web bash</code>	Open terminal in web container
<code>docker-compose exec web python manage.py migrate</code>	Run Django commands

7 Django + PostgreSQL + Redis Setup

📁 Project Structure

STRUCTURE

```

my_django_project/
├── myapp/
├── myproject/
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── Dockerfile
├── docker-compose.yml
├── .env
├── .dockerignore
├── manage.py
└── requirements.txt

```

🔑 .env File

.ENV

```

DEBUG=True
SECRET_KEY=your-secret-key-here
DATABASE_URL=postgresql://myuser:mypassword@db:5432/mydb
REDIS_URL=redis://redis:6379/0

```

⚠ IMPORTANT NOTE

In docker-compose, services communicate using their service NAME as hostname. So Django connects to PostgreSQL using hostname "db" (not localhost), and Redis using hostname "redis".

⚙️ Django settings.py — Database & Cache

PYTHON

```
import os

# Database — connects to 'db' service in docker-compose
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('POSTGRES_DB', 'mydb'),
        'USER': os.environ.get('POSTGRES_USER', 'myuser'),
        'PASSWORD': os.environ.get('POSTGRES_PASSWORD', 'mypassword'),
        'HOST': 'db',          # Service name in docker-compose
        'PORT': '5432',
    }
}

# Cache — connects to 'redis' service
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.redis.RedisCache',
        'LOCATION': 'redis://redis:6379/1',
    }
}
```

🔧 First Time Setup Commands

BASH

```
# Build and start all containers
docker-compose up --build

# In a new terminal — run migrations
docker-compose exec web python manage.py migrate

# Create superuser
docker-compose exec web python manage.py createsuperuser

# Collect static files
docker-compose exec web python manage.py collectstatic

# Open Django shell
docker-compose exec web python manage.py shell
```

8 Production Setup — Nginx + Gunicorn

In development, we use Django's built-in server. In production, we use **Gunicorn** (WSGI server) behind **Nginx** (reverse proxy).

📄 Production docker-compose.yml

DOCKER-COMPOSE.PROD.YML

```
version: '3.8'

services:
  web:
    build: .
    command: gunicorn myproject.wsgi:application --bind 0.0.0.0:8000
    volumes:
      - static_volume:/app/staticfiles
      - media_volume:/app/media
```

```

    env_file:
    - .env.prod
    depends_on:
    - db

  nginx:
    image: nginx:alpine
    ports:
    - "80:80"
    volumes:
    - ./nginx.conf:/etc/nginx/conf.d/default.conf
    - static_volume:/app/staticfiles
    depends_on:
    - web

  db:
    image: postgres:15
    env_file:
    - .env.prod

volumes:
  postgres_data:
  static_volume:
  media_volume:

```

nginx.conf

```

upstream django {
    server web:8000;
}

server {
    listen 80;

    # Static files served by Nginx (faster)
    location /static/ {
        alias /app/staticfiles/;
    }

    # Everything else goes to Django/Gunicorn
    location / {
        proxy_pass http://django;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_redirect off;
    }
}

```

9 Intern Practice Checklist

COMPLETE THESE TASKS TO MASTER DOCKER

Work through these in order. Each builds on the previous one.

Phase 1 — Basics

- Install Docker Desktop and run `docker run hello-world` successfully
- Pull official images: `docker pull python:3.11-slim`, `docker pull postgres:15`
- Run a container with port mapping: `docker run -p 80:80 nginx` and open in browser
- Practice `docker ps`, `docker stop`, `docker rm` commands

- Enter a running container with `docker exec -it container_id bash`

Phase 2 — Dockerfile

- Write a Dockerfile for your Django project from scratch
- Build the image with `docker build -t mydjango .`
- Run the container with `docker run -p 8000:8000 mydjango`
- Create a `.dockerignore` file
- Understand layer caching — change code vs change requirements.txt

Phase 3 — Docker Compose

- Write `docker-compose.yml` with Django + PostgreSQL + Redis
- Use `.env` file for all secrets and config
- Run `docker-compose up --build` and access your app
- Run migrations: `docker-compose exec web python manage.py migrate`
- Verify data persists after `docker-compose down` and `docker-compose up`

Phase 4 — Production

- Replace `runserver` with `Gunicorn` in your Compose file
- Add `Nginx` service and configure `nginx.conf`
- Serve static files through `Nginx`
- Deploy to a cloud server (`DigitalOcean` or `AWS EC2`)